

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA - UTN

Unidad: De Java a Python: POO por Equivalencia

APUNTE ACTIVIDAD 2

La property y el encapsulamiento

Índice

2 La property

Donde Python te libera de un dogma Java

- Por qué Java te enseñó a escribir getters
- La property: cambiar la implementación sin cambiar la interfaz
- El punto arquitectónico
- La regla, invertida

3 Encapsulamiento

De garantía del compilador a contrato de equipo

- El caso del final
- Una objeción honesta

2 La property

Donde Python te libera de un dogma Java

Este es **el cambio de hábito más importante** de todo el manual. Si te llevás una sola cosa, que sea esta.

Por qué Java te enseñó a escribir getters

Vale la pena empezar reconociendo que el hábito del getter, en Java, **es correcto**. No es superstición ni burocracia: es defensa propia, y responde a un problema real.

Supongamos que exponés `public int nroLados` y medio sistema escribe `figura.nroLados`. Un día descubrís que ese número tiene que calcularse, o validarse, o registrarse en un log. Necesitás lógica. Y en Java, meter lógica implica convertirlo en método, lo que implica que todos los clientes pasen a escribir `figura.getNroLados()`. Cambió la sintaxis de acceso: todo lo que dependía de esa clase deja de compilar. Si la clase es parte de una API pública, acabás de romper a todos tus usuarios.

El getter preventivo resuelve eso. Escribís `getNroLados()` desde el día uno, aunque hoy solo haga `return lados.size()`, porque así el día que necesite lógica la agregás adentro y nadie se entera. El getter es **un seguro contra el futuro**. El precio es escribir -y leer- miles de métodos que no hacen nada. La comunidad Java aceptó ese precio, con razón, porque la alternativa era peor.

Python no tiene ese problema, y por lo tanto no necesita ese seguro.

La property: cambiar la implementación sin cambiar la interfaz

En Python podés convertir un atributo en método sin que ningún cliente lo note. Empezás con `self.nro_lados = 3`; el día que necesita lógica, lo convertís en `@property`. El código que hacía `figura.nro_lados` sigue funcionando exactamente igual, sin recompilar, sin tocar una línea. La sintaxis de acceso es la misma para un atributo y para una property: **el cliente nunca supo la diferencia y nunca la va a necesitar saber**.

Esto da vuelta la regla. El getter preventivo dejó de comprar nada, porque el riesgo del que te protegía ya no existe. Y lo que antes era prudencia ahora es ruido.

Cambiar la implementación sin romper al cliente

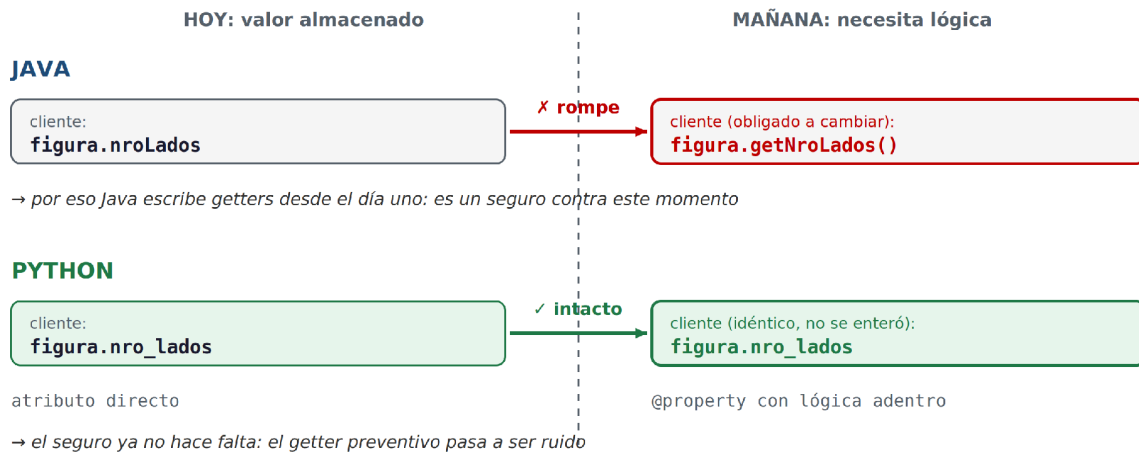


Figura 4 · El momento contra el que el getter Java asegura. En Python ese momento no rompe nada: el cliente escribe lo mismo antes y después.

El mismo modelo, dos implementaciones

Tomado del Ejercicio 1 de la serie - **Poligono**, con el atributo derivado `/nroLados` y la restricción refinada por subclase. Es el mismo diagrama en los dos manuales; lo único que cambia es el código:

JAVA

```
public abstract class Poligono extends Figura {
    private final List<Lado> lados = new ArrayList<>();
    public void agregarLado(Lado lado) { lados.add(lado); }
    /** Atributo derivado: se calcula, no se almacena. */
    public int nroLados() { return lados.size(); }
    /** Cada subclase concreta refina la cantidad exacta. */
    protected abstract int ladosEsperados();
    public boolean estructuraValida() {
        int esperados = ladosEsperados();
        if (esperados == -1) return nroLados() >= 3;
        return nroLados() == esperados;
    }
}
```

PYTHON

```
from abc import ABC, abstractmethod

class Poligono(Figura, ABC):

    def __init__(self) -> None:
        super().__init__()
        self._lados: list[Lado] = []

    def agregar_lado(self, lado: Lado) -> None:
        self._lados.append(lado)

    @property
    def nro_lados(self) -> int: # derivado: se calcula
        return len(self._lados)

    @abstractmethod
    def lados_esperados(self) -> int:
        """Cada subclase concreta refina la cantidad exacta."""

    @property
    def estructura_valida(self) -> bool:
        esperados = self.lados_esperados()
        if esperados == -1:
            return self.nro_lados >= 3
        return self.nro_lados == esperados
```

Mirá qué se conservó y qué se movió. La **jerarquía** es la misma. El **método abstracto** que cada subclase refina es el mismo. La **lógica de validación** es idéntica, línea por línea. Lo único que cambió de naturaleza es `nroLados()`, que era un método y pasó a ser algo que se *lee* como atributo.

Una aclaración sobre el encabezado, porque genera una pregunta legítima en clase: `class Poligono(Figura, ABC)` lleva `ABC` para que el ejemplo funcione aislado. Si `Figura` ya hereda de `ABC` - como ocurre en el manual de la serie- alcanza con `class Poligono(Figura);`, porque la condición para que `@abstractmethod` tenga efecto es que haya un `ABC` en algún punto de la cadena de herencia, no en el encabezado inmediato. Las dos formas son correctas; lo que no funciona es `@abstractmethod` en una clase sin ningún `ABC` en su ascendencia: ahí el decorador se ignora en silencio y la clase se deja instanciar incompleta.

El punto arquitectónico

`figura.nro_lados` se lee como acceso a campo pero ejecuta un método. Y acá aparece algo que vale más que una comodidad sintáctica: en el diagrama UML, `/nroLados` está marcado como **atributo**

derivado. La barra inclinada significa exactamente eso: es un atributo -se lee como atributo, se usa como atributo- cuyo valor se calcula en lugar de almacenarse.

En Java, ese atributo derivado del modelo se convierte necesariamente en un método. Pierde su cara de atributo en la traducción. El lector del código Java ve `figura.nroLados()` y tiene que reconstruir mentalmente que eso, en el modelo, era un /atributo. En Python conserva la cara: `figura.nro_lados`.

La sintaxis Python está más cerca del UML que la Java. El / del diagrama se corresponde con `@property`. Esto no es un detalle estético: significa que el código Python conserva una distinción que el modelo hacía y que el código Java pierde. Cuando enseñás atributos derivados con el manual Java, tenés que explicar que el / «se implementa como método». Con el manual Python, el / se implementa como lo que es.

La regla, invertida

En Java	En Python
Escribí getters siempre, por las dudas.	Exponé el atributo directo. Ponés <code>@property</code> el día que necesitás lógica.

Escribir getters preventivos en Python es un java-ismo, y el code reviewer te lo va a marcar. No es cuestión de estilo. El argumento de fondo es que **el ruido oculta la señal**: si todos los accesos pasan por un `get_x()` idéntico, el lector no tiene forma de saber cuáles tienen lógica detrás y cuáles no, y termina no leyendo ninguno. Cuando solo los que hacen algo son properties, el `@property` se vuelve informativo - dice «acá pasa algo, prestá atención». Un decorador que está en todos lados no informa nada.

Cuándo sí escribir una property

La regla «no escribas getters» no significa «nunca uses properties». La property se justifica cuando hay algo que decir:

- **Atributo derivado.** El caso de `nro_lados`: el valor se calcula a partir de otros. Es el uso canónico y el que aparece en el modelo UML con la barra.
- **Validación al asignar.** Un `@x.setter` que rechaza valores inválidos. Es donde va la restricción del enunciado que no se dibuja como caja.
- **Solo lectura.** Una property sin setter es el equivalente más cercano al campo final expuesto: el cliente lee, no escribe.
- **Cálculo costoso que conviene cachear.** Con `@cached_property`, el valor se computa una vez y queda. En Java esto es un campo más lazy init a mano.

Si ninguna de esas cuatro aplica, el atributo va directo y punto. Y el día que aplique alguna, la property entra sin romper a nadie - que es, después de todo, el argumento entero de este capítulo.

Así se ve «el día que necesitás lógica». Supongamos que Lado empezó exponiendo `self.longitud` directo, y el enunciado agrega la restricción de que un lado no puede medir cero o menos - una de esas reglas que en el diagrama van como nota entre llaves, no como caja:

PYTHON

```
class Lado:

    def __init__(self, longitud: float) -> None:
        self.longitud = longitud # pasa por el setter de abajo

    @property
    def longitud(self) -> float:
        return self._longitud

    @longitud.setter
    def longitud(self, valor: float) -> None:
        if valor <= 0:
            raise ValueError("un lado debe medir más que cero")
        self._longitud = valor
        \# El cliente escribe lo mismo que siempre:
        lado.longitud = 5.0 # ✓ valida y asigna
        lado.longitud = -1 # ✗ ValueError - la regla del enunciado
```

Fijate dos cosas. Primero, el cliente que hacía `lado.longitud = 5.0` antes de que existiera la validación escribe exactamente lo mismo después: la restricción entró sin romper a nadie. Segundo, hasta el propio `__init__` asigna a través del setter, así que la regla se aplica también al construir - no hay puerta trasera. Este es el destino natural de las restricciones del enunciado que el manual de la serie enseña a no dibujar como cajas: terminan viviendo en un setter o en un raise del constructor.

3 Encapsulamiento

De garantía del compilador a contrato de equipo

Esta es la mesa donde más incomodidad vas a sentir, y conviene atravesarla de frente en vez de buscarle un atajo. Python tiene tres niveles de visibilidad y **ninguno** es lo que Java llama private. No hay un equivalente escondido ni un truco que lo reproduzca. Simplemente no está.

PYTHON

```
class Cuenta:

    def __init__(self) -> None:
        self.saldo = 0 # público
        self._interno = 0 # "protected" - convención
        self.__mangled = 0 # name mangling
```

- `_interno`: **nada** te impide tocarlo desde afuera. Es un cartel de «no entres». Los linters lo marcan, los IDEs lo esconden del autocompletado, la comunidad lo respeta; el intérprete lo ignora.
- `__mangled`: se renombra a `_Cuenta__mangled`. **No es privado** - es antichoque de nombres en herencia, un mecanismo pensado para que una subclase no pise sin querer un atributo interno del padre. `c._Cuenta__mangled` funciona perfecto desde cualquier lado.

El error frecuente es usar el doble guión creyendo que es el private de Java. No lo es, y además tiene un costo: rompe la herencia legítima -una subclase no puede acceder al atributo del padre sin escribir el nombre mangleado- y complica el debugging y los tests. La convención de la comunidad es clara: **usá un guión bajo simple**. El doble se reserva para el caso puntual de evitar colisiones en jerarquías profundas, que es para lo que fue diseñado.

Vale la pena ver ese caso puntual una vez, para que el mecanismo deje de ser misterioso. El mangling existe para que padre e hijo puedan usar el mismo nombre interno sin pisarse:

PYTHON - PARA QUÉ EXISTE EL DOBLE GUIÓN

```
class Figura:

    def __init__(self) -> None:
        self.__cache = None # se guarda como _Figura__cache

class Poligono(Figura):

    def __init__(self) -> None:
        super().__init__()
        self.__cache = {} # se guarda como _Poligono__cache
        \# Sin mangling, la asignación del hijo pisaría la del padre.
        \# Con mangling, conviven: son dos atributos distintos.
```

Eso es todo lo que el doble guión hace: renombrar por clase para que no haya choque. Leído así queda claro por qué usarlo «para privacidad» es un error de concepto - el renombre es predecible y público, no una barrera.

Estructural vs. cultural

JAVA · el lenguaje impone



la barrera está en el lenguaje
cruzarla = no compila

PYTHON · el equipo acuerda



la barrera está en la cultura
cruzarla = te lo marcan

La decisión de diseño —qué es interno— es exactamente la misma. Cambia quién la hace cumplir.

Figura 5 · La misma frontera, dos mecanismos: en Java es un muro del lenguaje; en Python, un cartel que el equipo y sus herramientas hacen respetar.

El caso del final

El `private final List<Lado> lados` del Ejercicio 1 dice dos cosas a la vez, y conviene separarlas porque Python las trata distinto. Dice «nadie de afuera toca esta lista» (`private`) y dice «esta referencia no se reasigna nunca» (`final`). Notá que el `final` de Java protege la *referencia*, no el *contenido*: `lados.add(x)` compila perfectamente en un campo `final`. Es una garantía más débil de lo que parece.

Python ofrece tres aproximaciones, de menos a más fuerte:

PYTHON

```
\# 1. Documentar la intención (mypy lo verifica, el runtime no)
from typing import Final
self._lados: Final[list[Lado]] = []
\# 2. Property de solo lectura → protege la referencia, no el contenido

@property
def lados(self) -> list[Lado]:
    return list(self._lados) # copia defensiva: el cliente no muta
\# 3. Inmutabilidad real
from dataclasses import dataclass

@dataclass(frozen=True)
class Lado:
    longitud: float
```

La primera es pura documentación: Final le dice a mypy que no querés reasignar, y mypy te avisa si lo hacés. En ejecución no pasa nada. Es el equivalente honesto del final de Java en cuanto a fuerza real de la garantía, y de hecho es más útil, porque mypy te avisa en el editor.

La segunda es la más importante en la práctica y la que más conviene incorporar: **property de solo lectura que devuelve una copia**. Es más fuerte que el final de Java, porque el final dejaba que el cliente hiciera getLados().add(x) si el getter devolvía la lista interna. Acá el cliente recibe una copia: puede mutarla todo lo que quiera y el objeto no se entera. La lista interna sigue intacta.

La tercera es la única inmutabilidad real, y es más fuerte que cualquier cosa que Java te dé sin escribir código: frozen=True hace que asignar a un campo tire FrozenInstanceError. Sirve para los objetos-valor del modelo -Lado, Punto, Medida-, esos que se comparan por contenido y no tienen identidad propia. Ese criterio, el de *value object* contra *entity*, es el mismo que aplicabas en Java.

TRADUCCIÓN PARA UN ARQUITECTO

En Java el encapsulamiento es **estructural** (lo impone el lenguaje). En Python es **cultural** (lo impone el equipo, el linter y el code review). El diseño no cambia: cambia quién lo hace cumplir.

Si venís de Java, tu instinto de encapsular está bien y no hay que apagarlo. Solo que ahora la **property de solo lectura con copia defensiva** es tu herramienta principal, no el modificador. Y la segunda herramienta -la que la mayoría no instala- es mypy corriendo en CI: sin él, Final es un comentario decorado.

Una objeción honesta

Llegado acá, el programador Java suele plantear la objeción obvia: si nada lo impide, ¿no es todo esto una ficción? ¿No es Python simplemente menos seguro?

La respuesta tiene dos partes. La primera es que el `private` de Java tampoco es absoluto: existe la reflexión, existe `setAccessible(true)`, y cualquier framework de serialización o de inyección de dependencias los usa todos los días para leer y escribir campos privados de tus clases. La barrera es alta, no infinita.

La segunda parte es más interesante. El encapsulamiento nunca fue, en el fondo, un mecanismo de seguridad: es un mecanismo de **comunicación**. Sirve para decirle al próximo programador -que muchas veces sos vos en seis meses- qué partes del objeto son contrato estable y qué partes son implementación que puede cambiar. Un atacante no es el destinatario del `private`; un colega lo es. Y para comunicar, un `_` que la comunidad entera entiende, que el linter marca y que el reviewer señala, comunica exactamente lo mismo que un `private`.

Lo que Python te quita no es el encapsulamiento: es la posibilidad de delegarlo. En Java podías no pensar en el tema y dejar que el compilador se encargara. En Python tenés que decidirlo, escribirlo y sostenerlo. Es más trabajo, y también es más honesto respecto de lo que el encapsulamiento siempre fue.